SS

FRONTEND GUIDE · REACT + VITE

دليل الفرونت إند
بورتفوليو سالم صالح

شرح مبسّط لكل جزء في واجهة الموقع: إزاي متقسّمة، وإزاي البيانات بتوصل من لوحة التحكم للزائر، والمفاهيم الأساسية في React بأمثلة من كودك انت – عشان تقدر تفهم أي ملف وتضيف ميزة جديدة بنفسك.

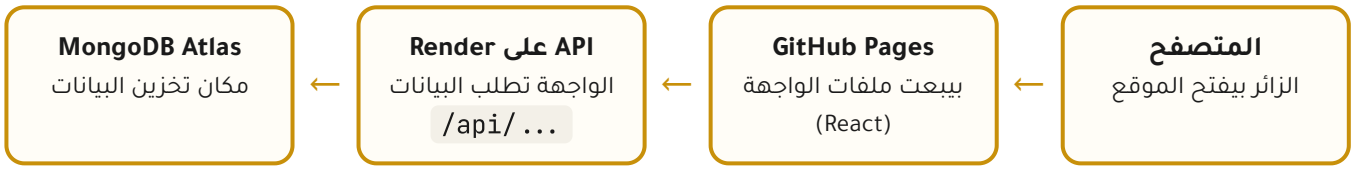
1. الصورة الكبيرة
2. تشغيل المشروع على جهازك
3. خريطة الملفات
4. مفاهيم React من كودك
5. رحلة البيانات من الداشبورد للزائر
6. اللغتين والثيم
7. الأقسام واحد واحد
8. لوحة التحكم (Admin)
9. الحركة عند النزول (useReveal)
10. إضافة ميزة جديدة
11. النشر
12. مشاكل حصلت واتعلمنا منها
13. قاموس المصطلحات

1 الصورة الكبيرة

الموقع عبارة عن **تطبيق React واحد** (Single Page Application) فيه جزئين:

- **الموقع العام** على الرابط `/` – اللي الزوار بيشوفوه: Hero، About، الخدمات، المهارات، المشاريع، الآراء، يوتيوب، التواصل.
- **لوحة التحكم** على الرابط `/#/admin` – اللي انت بس بتدخلها بالإيميل والباسورد وبتعدّل منها كل محتوى الموقع.

الواجهة نفسها **ملفات ثابتة** (HTML + CSS + JavaScript) متنشورة على **GitHub Pages**. أما البيانات (المشاريع، المهارات، الآراء...) فمش جوه الكود – بتيجي من **السيرفر (API)** على Render، واللي بيقرا ويكتب في قاعدة البيانات **MongoDB Atlas**.



> **القاعدة الأهم في المشروع**
>
> أي محتوى بيظهر للزائر لازم يتعدّل من لوحة التحكم – مش من الكود. عشان كده كل قسم بيجيب بياناته من الـ API، ولو مفيش بيانات بيرجع لقيم افتراضية أو بيختفي.

الأدوات المستخدمة

الأداة	دورها
React 18	مكتبة بناء الواجهة من "مكوّنات" (Components) صغيرة.
Vite	أداة التشغيل والبناء: `npm run dev` للتطوير، و `npm run build` لإنتاج ملفات النشر في `dist/`.
React Router (HashRouter)	التنقل بين الصفحات (`/` و `/admin/...`) من غير ما الصفحة تعيد التحميل.
CSS عادي	كل الستايل في ملف واحد `src/index.css` بمتغيرات ألوان للثيمين.

2 تشغيل المشروع على جهازك

المشروع جزئين بيشتغلوا مع بعض، فمحتاج **اتنين Terminal**:

```
# Terminal 1 – السيرفر (يكون شغّال على جهازك لازم MongoDB)
cd "D:\Work\Web applications\My-portfolio-\My-portfolio-\server"
npm install       # أول مرة بس
npm run dev       # API on http://localhost:4000

# Terminal 2 – الواجهة
cd "D:\Work\Web applications\My-portfolio-\My-portfolio-\client"
npm install       # أول مرة بس
npm run dev       # افتح http://localhost:5173
```

وقت التطوير، أي طلب من الواجهة لـ `/api/...` بيتحوّل تلقائيًا للسيرفر على بورت 4000 – ده إعداد اسمه **proxy** موجود في `client/vite.config.js`. وفي النسخة المنشورة، الواجهة بتكلم السيرفر على Render من خلال الرابط المكتوب في `client/.env.production`.

> **نصيحة**
>
> لو شغّلت السيرفر وطلع خطأ `EADDRINUSE :::4000` يبقى فيه نسخة تانية من السيرفر شغّالة على نفس البورت – اقفلها الأول.

3 خريطة الملفات

```
client/
├── index.html              ← الصفحة الوحيدة: فيها SEO وصورة المشاركة والثيم الافتراضي
├── vite.config.js          ← base = /My-portfolio/ + الـ proxy للسيرفر
├── .env.production         ← VITE_API_URL: رابط السيرفر على Render
├── public/                 ← ملفات بتتنسخ زي ما هي (صورتك، og-image.jpg)
└── src/
    ├── main.jsx            ← نقطة البداية: بيركّب App جوه الصفحة
    ├── App.jsx             ← كل الروابط (Routes) + الـ Providers
    ├── index.css           ← كل الستايل وألوان الثيمين
    ├── pages/
    │   ├── Portfolio.jsx   ← الموقع العام: بيرتّب الأقسام
    │   ├── FeedbackForm.jsx ← فورم رأي العميل /#feedback/ar|en
    │   └── admin/          ← صفحات لوحة التحكم + FormParts.jsx
    ├── components/         ← الأقسام: Hero, About, Services, Skills, Projects, ...
    ├── hooks/              ← منطق قابل لإعادة الاستخدام: useReveal, useCollection, ...
    ├── context/            ← حالة عامة: SiteContext (لغة/ثيم), AdminContext (الدخول)
    ├── lib/                ← أدوات JavaScript عادية: api.js, apiCache.js, pick.js, ...
    └── i18n/translations.js ← نصوص الواجهة الثابتة بالعربي والإنجليزي
```

الفرق بين `hooks/` و `lib/` و `components/`

المجلد	فيه إيه	مثال
`components/`	حاجات بتترسم على الشاشة (بترجع JSX)	`Hero.jsx` بيرسم أول جزء في الصفحة
`hooks/`	دوال React بتبدأ بـ `use` وبتستخدم state أو effects	`useCollection("/projects")` بيجيب المشاريع
`lib/`	JavaScript عادي من غير React	`api.js` لكل طلبات السيرفر، `pick.js` لاختيار اللغة
`context/`	بيانات محتاجها الموقع كله	`useSite()` بيدّيك اللغة والثيم والترجمة

4 مفاهيم React من كودك

1) المكوّن (Component) و JSX

المكوّن = دالة بترجع شكل جزء من الصفحة. الشكل ده مكتوب بـ **JSX** – شبه HTML جوه JavaScript. مثال حقيقي مبسّط من `Footer.jsx`:

client/src/components/Footer.jsx

```
export default function Footer({ settings }) {
  const { t, lang } = useSite();
  return (
    <footer>
      <div className="footer-name">{pick(lang, settings.nameEn, settings.nameAr)}</div>
      <div className="footer-copy">{t.footer.rights}</div>
    </footer>
  );
}
```

لاحظ: `className` بدل `class`، وأي قيمة JavaScript بتتحط بين `{ }`.

2) الـ Props – بيانات داخلة للمكوّن

الـ props هي "المعطيات" اللي المكوّن الأب بيبعتها للابن. في `Portfolio.jsx` بنجيب إعدادات الموقع مرة واحدة ونوزّعها:

```
<Hero settings={settings} />
<Contact settings={settings} />
<Footer settings={settings} />
```

3) الـ State – بيانات بتتغير وبتعيد الرسم

`useState` بيحفظ قيمة جوه المكوّن، ولما تتغير React يعيد رسم المكوّن. مثال من فورم التواصل:

client/src/components/Contact.jsx

```
const [status, setStatus] = useState("idle"); // idle | sending | success | error
// ...
setStatus("sending");
await api.post("/contact", formData);
setStatus("success");    // الشاشة تتحدّث لوحدها وتظهر رسالة النجاح
```

4) الـ Effect – حاجة تحصل بعد الرسم

`useEffect` للحاجات اللي "برّه" الرسم: طلب بيانات من السيرفر، تايمر، الاستماع للـ scroll. مثال: تبديل المسميات في الـ Hero كل 3.2 ثانية:

```
useEffect(() ⇒ {
  if (count < 2) return;                    // مفيش حاجة تتبدّل
  const cycle = setInterval(() ⇒ { /* المسمى اللي بعده */ }, 3200);
  return () ⇒ clearInterval(cycle);         // تنضيف لما المكوّن يختفي
}, [count]);                                // يتعاد لو عدد المسميات اتغير
```

5) Custom Hook – منطق بتعيد استخدامه

لو نفس المنطق متكرر في أكتر من مكوّن، بنحطه في دالة اسمها يبدأ بـ `use`. أشهر مثال في المشروع `useCollection`: بيجيب قايمة من السيرفر ويرجّع `{ data, loading }`:

```
const { data: projects } = useCollection("/projects");
```

6) Context – بيانات للموقع كله

بدل ما نبعت اللغة والثيم كـ props لكل مكوّن، `SiteContext` بيخزّنهم مرة واحدة فوق كل حاجة (في `App.jsx`)، وأي مكوّن يقدر يقراهم بـ:

```
const { t, lang, theme, toggleLang, toggleTheme } = useSite();
```

7) العرض المشروط والقوايم

```
// لو مفيش خدمات، القسم كله ما يترسمش
if (loading || items.length === 0) return null;

// قايمة: لازم key فريد لكل عنصر
{items.map((item) ⇒ <div className="service-card" key={item.id}>...</div>)}
```

5 رحلة البيانات من الداشبورد للزائر

خلينا نتابع "إضافة خدمة جديدة" من أولها لآخرها:

1. **انت في الداشبورد** (`AdminServices.jsx`) تضغط `api.post("/admin/services", ...)` → Save.
2. `lib/api.js` بيضيف التوكن بتاعك (`Authorization: Bearer ...`) ويبعت الطلب للسيرفر.
3. **السيرفر** يتأكد إنك الأدمن، ويتحقق من البيانات، ويحفظها في MongoDB.
4. **زائر يفتح الموقع:** `Portfolio.jsx` بينادي `useCollection("/services")` → طلب `GET` `/api/services`.
5. **القسم** `Services.jsx` بياخد القايمة ويرسم الكروت.

lib/api.js – باب واحد لكل الطلبات

أي كلام مع السيرفر لازم يعدّي من `api.get / post / put / patch / del` – مش `fetch` مباشرة. الملف ده بيعمل 3 حاجات مرة واحدة: يحدد رابط السيرفر، يضيف التوكن لو انت مسجّل دخول، ويرمي خطأ واضح لو السيرفر رد بغلط.

الكاش – المحتوى يظهر فورًا للزائر الراجع

السيرفر المجاني على Render ممكن ينام، وأول طلب بعدها ياخد وقت. عشان كده `lib/apiCache.js` بيحفظ آخر رد في متصفح الزائر (localStorage). الزيارة الجاية: المحفوظ يظهر **فورًا**، وبعدين يتحدّث بالجديد من السيرفر.



لوحة التحكم ما بتستخدمش الكاش – لازم تشوف البيانات الحقيقية دايمًا.

6 اللغتين والثيم

نوعين من النصوص

النوع	مكانه	مثال
نصوص الواجهة (ثابتة)	`t.contact.send` ← `i18n/translations.js`	"Send Message" / "إرسال"
نصوص المحتوى (من الداشبورد)	كل حقل له نسختين في الداتا: `titleEn` و `titleAr`	عنوان مشروع، نبذة About

لاختيار النسخة المناسبة من المحتوى بنستخدم `pick` من `lib/pick.js` :

```
pick(lang, item.titleEn, item.titleAr)
// عربي؟ ← titleAr. ولو فاضي يرجع titleEn (والعكس) – عمره ما يظهر فاضي
```

ولما اللغة عربي، `SiteContext` بيضيف class اسمه `ar` على `<body>` ، والـ CSS بيقلب الاتجاه (RTL) ويغيّر الخط لـ Tajawal.

> **قاعدة**
>
> أي نص ثابت جديد في الواجهة: ضيفه في `T.en` و `T.ar` بنفس المفتاح. أي حقل محتوى جديد: نسختين En و Ar.

الثيم (فاتح / داكن)

- الألوان كلها متغيرات في أول `:root` : `index.css` للداكن (رمادي غامق `#141414`) و `:root.light` للفاتح.
- الافتراضي **فاتح**: `<html class="light">` في `index.html` .
- اختيار الزائر بيتحفظ في `localStorage` ، وسكريبت صغير في `index.html` بيطبّقه قبل ما React يشتغل – عشان الصفحة ما "تومضش" بالثيم الغلط.

7 الأقسام واحد واحد

القسم	الملف	البيانات منين	بتتعدّل من
Hero	`Hero.jsx`	`GET /settings`	Profile & Links
About + فيديو	`About.jsx`	`GET /about` + `/cv/info`	About + CV
الخدمات	`Services.jsx`	`GET /services`	Services
المهارات	`Skills.jsx`	`GET /skills`	Skills
المشاريع + دراسة الحالة	`Projects.jsx` + `ProjectModal.jsx`	`GET /projects`	Projects
الشهادات	`Certificates.jsx`	`GET /certificates` + صورة كل شهادة	Certificates
آراء العملاء	`Testimonials.jsx`	`GET /testimonials`	Testimonials
يوتيوب	`YouTube.jsx`	`GET /youtube` + settings	YouTube + Profile & Links
التواصل	`Contact.jsx`	`POST` + (روابط) settings `/contact`	Profile & Links / Messages
الفوتر	`Footer.jsx`	settings	Profile & Links

تفاصيل تستاهل تعرفها

- **الأقسام الاختيارية** (الخدمات، الشهادات، الآراء) بتختفي لو فاضية – وروابطها في الـ Navbar كمان. عشان كده `Portfolio.jsx` هو اللي بيجيب بياناتها وبيديها للقسم وللـ Navbar.
- **فلتر المشاريع** أزراره مش مكتوبة في الكود: بتتبني من حقل Tags في المشاريع نفسها، الأكتر استخدامًا الأول.
- **نافذة المشروع** بتعرض "المشكلة ← الحل ← النتيجة" لو موجودين، وأزرار Demo/Code بتظهر بس لو الرابط حقيقي (يبدأ بـ http).
- **الصور والملفات** في `public/` لازم رابطها يتبني بـ `import.meta.env.BASE_URL` عشان الموقع شغّال تحت `/My-portfolio-/`.

8 لوحة التحكم (Admin)

تسجيل الدخول خطوة بخطوة

1. تفتح `/#/admin/login` وتكتب الإيميل والباسورد (`AdminLogin.jsx`).
2. `AdminContext.jsx` يبعت `POST /auth/login` – ولو صح يرجع **توكن** (JWT) صالح 7 أيام.
3. التوكن بيتحفظ في `localStorage` باسم `adminToken` – فال Refresh ما بيخرّجكش.
4. `api.js` بيضيف التوكن لكل طلب، والسيرفر يرفض أي طلب أدمن من غيره.
5. `AdminLayout.jsx` هو "الحارس": لو مفيش توكن يحوّلك لصفحة الدخول.

نمط صفحات الأدمن

أغلب الصفحات (Projects, Skills, Services, Testimonials, YouTube) نفس الشكل: **جدول** + **نافذة فورم** للإضافة والتعديل. الـ state المهم:

```
const [editing, setEditing] = useState(null); // null = مقفول | "new" = إضافة | id = تعديل
const [form, setForm] = useState(EMPTY);      // قيم الفورم الحالية
const load = () ⇒ api.get("/admin/services").then(setItems); // إعادة تحميل بعد أي تعديل
```

والصفحات اللي بتعدّل "مستند واحد كبير" (About و Profile & Links) بتستخدم مكوّنات مشتركة من `pages/admin/FormParts.jsx`: `Field` (حقل نص) و`ListEditor` (قايمة بإضافة/حذف/ترتيب).

9 الحركة عند النزول (useReveal)

العناصر اللي عليها `data-ao` بتبدأ **شفافة** (`opacity: 0` :CSS)، ولما توصل لها وانت نازل بتاخد class `visible` وتظهر بحركة.

```
const [headerRef, headerVisible] = useReveal();
<div ref={headerRef} className={`section-header ${headerVisible ? "visible" : ""}`} data-ao>
```

جوه `useReveal` بيستخدم `IntersectionObserver` – خاصية في المتصفح بتقول "العنصر ده بقى ظاهر في الشاشة".

درس اتعلمناه

قبل كده كان `useReveal` بيراقب العنصر *مرة واحدة وقت التحميل*. فلو العنصر ظهر بعدين (مثلًا بعد ما البيانات وصلت) ما حدش كان بيراقبه، فيفضل شفاف للأبد – وده اللي خلّى قسم About "يختفي" مرة.

الحل: `ref` بقى "callback ref" محفوظ في state، فأي عنصر يظهر في أي وقت بيتراقب.

تحذير

أي عنصر بتحط عليه `data-ao` لازم يكون معاه `ref` من `useReveal` ويتغيّر له `visible` class – وإلا هيفضل مخفي.

10 إضافة ميزة جديدة

كل ميزة في الموقع بتتبني بنفس الخمس خطوات (قاعدة "كل حاجة من الداشبورد"). مثال: قسم "الشهادات":

1. **الموديل** في السيرفر: `server/src/lib/mongo.js` (الحقول بنسختين En/Ar).
2. **API عام** للقراءة: `server/src/routes/certificates.js` + سطر في `app.js` .
3. **API للأدمن** (إضافة/تعديل/حذف): قسم جديد في `routes/admin.js` .
4. **صفحة أدمن**: `pages/admin/AdminCertificates.jsx` (انسخ نمط Route + `AdminServices.jsx`) في `App.jsx` + رابط في `AdminLayout.jsx` .
5. **قسم العرض**: `components/Certificates.jsx` + إضافته في `Portfolio.jsx` + العنوانين في `translations.js` + الستايل في `index.css` .

> **قبل ما تبدأ**
>
> اعمل branch جديد (`git checkout -b feat/certificates`). ولما تخلص: جرّب على جهازك → `npm run build` لازم ينجح → وبعدين انشر.

11 النشر

بناء النسخة النهائية

```
cd client
npm run build       # يطلّع ملفات الموقع في client/dist
```

Vite بيقرا `VITE_API_URL` من `.env.production` ويحطه جوه الكود، وبيظبط كل الروابط على `/My-portfolio-/` (من `vite.config.js`).

النشر على GitHub Pages – الطريقة الحالية

GitHub Actions متوقف حاليًا (مشكلة في الفوترة على الحساب)، فالموقع بيتنشر من branch اسمه `gh-pages` فيه محتوى `dist/` بس:

```
npm run build
# انسخ محتوى client/dist لمجلد مؤقت فيه نسخة من branch gh-pages
# + ملف فاضي اسمه .nojekyll، وبعدين:
git add -A && git commit -m "deploy" && git push origin gh-pages
```

لما Actions يرجع يشتغل: **GitHub Actions** = Source ← Pages ← Settings، وساعتها أي push على `main` بينشر لوحده (`.github/workflows/deploy-pages.yml`).

> **بعد النشر**
>
> افتح الموقع بـ `Ctrl+Shift+R` أو نافذة Incognito – المتصفح بيحتفظ بالنسخة القديمة حوالي 10 دقايق.

12 مشاكل حصلت واتعلمنا منها

المشكلة	السبب	الحل / الدرس
عناوين الأقسام مش ظاهرة وفراغ مكانها	`data-ao` من غير ما حد يضيف `visible`	كل عنصر `data-ao` لازم يكون له `useReveal`
قسم About اختفى بعد ما بقى من الداشبورد	`useReveal` بيراقب وقت التحميل بس، والمحتوى ظهر بعدين	callback ref – يراقب العنصر وقت ما يظهر
الصورة الشخصية مش ظاهرة على الموقع المنشور	الرابط `/uploads/...` من جذر الدومين، والموقع تحت `/My-portfolio/`	استخدم `import.meta.env.BASE_URL`
المحتوى بياخد دقيقة عشان يظهر	السيرفر المجاني بينام بعد 15 دقيقة	كاش في المتصفح + السيرفر يزور نفسه كل 10 دقايق
الموقع ما بيبنيش بعد تعديل لينك	اللصق في VS Code قطّع الأيقونة على أسطر	غيّر النص جوه علامات التنصيص بس، وجرّب `npm run build` قبل النشر
الموقع المنشور بيعرض README	Pages كان متظبط "من branch" بـ Jekyll	Pages Source الصح + ملف `.nojekyll`

13 قاموس المصطلحات

المصطلح	معناه ببساطة
Component	دالة بترجع جزء من الصفحة (JSX).
JSX	كتابة شبه HTML جوه JavaScript.
Props	بيانات داخلة لمكوّن من المكوّن اللي فوقه.
State	بيانات جوه المكوّن، لما تتغير الشاشة تتحدّث.
Hook	دالة React بتبدأ بـ use (useState, useEffect, useCollection...).
Effect	كود بيشتغل بعد الرسم (طلبات، تايمرات).
Context	مخزن بيانات عام لكل الموقع من غير props.
Router / Route	اختيار الصفحة اللي تظهر حسب الرابط.
API	عنوان على السيرفر بتطلب منه أو تبعتله بيانات.
JWT / Token	"تصريح دخول" موقّع بيثبت إنك الأدمن.
localStorage	تخزين صغير جوه متصفح الزائر بيفضل بعد قفل الصفحة.
Build	تحويل الكود لملفات جاهزة للنشر (`dist/`).
RTL	الكتابة من اليمين للشمال (العربي).
Cache	نسخة محفوظة من بيانات عشان تظهر أسرع.

الدليل ده بيوصف الكود في أكتوبر 2026. لو المشروع اتغيّر، الملف المصدر `docs/guides/frontend-guide.html` وتقدر تعدّله وتعيد طباعته بـ `docs/guides/build-pdfs.sh` .